

Behavioral Synthesis of ICs

High Level Synthesis Algorithms

Credentials: majority of information are from the following sources:

1. Mohanty, Saraju P., Nagarajan Ranganathan, Elias Kougianos, and Priyadarsan Patra. *Low-power high-level synthesis for nanoscale CMOS circuits*. Springer Science & Business Media, 2008.

Outline

- **Behavioral HDL to CDFG Translation or Compilation**
- **Scheduling Algorithms**
 - ✓ ASAP and ALAP Scheduling and Mobility
 - ✓ Integer Linear Programming (ILP) Scheduling
 - ✓ List-Based Scheduling (LBS)
 - ✓ Force-Directed Scheduling (FDS)
 - ✓ Game Theory Scheduling (GTS)
 - Tabu Search Scheduling (TSS)
 - Simulated Annealing Scheduling (SAS)
 - Genetic Algorithm Scheduling (GAS)
 - Ant Colony Scheduling (ACS) .
 - Automata-Based Symbolic Scheduling
 - ✓ Chaining, Multicycling and Pipelining Data Paths

Objectives

- Understand the process of translating behavioral hardware description language (HDL) descriptions into Control Data Flow Graphs (CDFG) as part of the behavioral synthesis workflow.
- Explore the various scheduling algorithms utilized in High-Level Synthesis (HLS) to optimally schedule operations for efficient hardware realization.



Behavioral HDL to CDFG Translation or Compilation

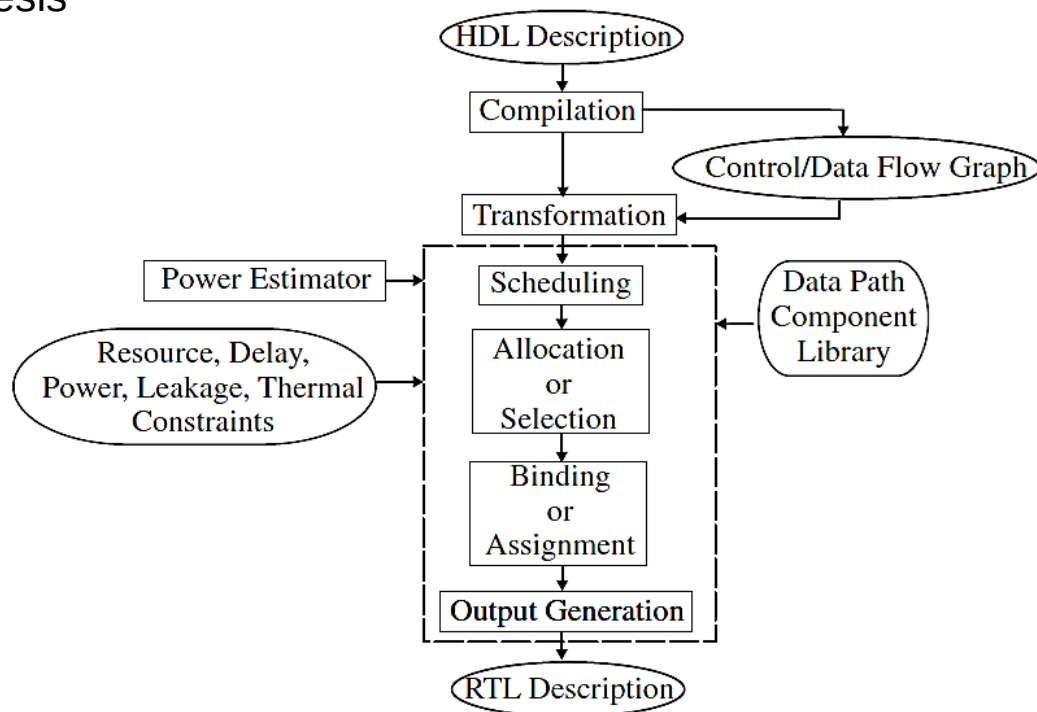
Transforming Behavioral HDL into precise, graph-based blueprints, paving the path for optimized digital synthesis and the physical implementation.

Various Phases of High-Level Synthesis

- The **high-level synthesis** process takes a system in the form of a **hardware description language** (HDL) as input and generates an **optimal RTL description**.
- The various phases or tasks of high-level synthesis include compilation, transformation, scheduling, allocation and binding as detailed in the next slide .
- Additional steps like **power optimization**, **leakage optimization**, **register optimization** and **interconnect optimization** also take place in synthesis either sequentially or along with the fundamental steps.
- Thus, various other inputs in the form of a **datapath component library** and **constraints** are needed for **trade-off** and **design space exploration** study.
- There is no fixed sequence for performing various high-level synthesis tasks. Although they are independent of each other, these tasks should be performed simultaneously for effective optimization.

Various Phases of High-Level Synthesis

→ Various phases of high-level synthesis



Compilation

- The behavior of a system to be synthesized is usually specified at the **algorithmic level** using a **high-level programming language** like C/C++ or a hardware description language (HDL) such as VHDL and Verilog.
- The behavior of the system is then compiled into internal representations, which are usually **data flow graphs (DFGs)** and **control flow graphs (CFGs)**.
- Each behavioral specification is transformed into a unique **graphical representation**.
- The **DFG** is a directed graph that represents **data movement**, whereas the **CFG** is a directed graph that indicates the **sequence of operations**.

Transformation

- In the **transformation** step, the **initial DFG** is transformed so that the **resultant DFG** is more suitable for **scheduling** and **allocation**.
- These transformations include **compiler-like optimizations** such as **dead-code elimination**, **common subexpression elimination**, **loop unrolling**, **constant propagation** and **code motion**.
- In addition, some **hardware-specific transformations** like minimization of syntactic variances and retiming may be applied to take advantage of the **associativity** and **commutativity** of certain operations.

Scheduling

- **Scheduling** is the process of **partitioning** the set of **arithmetic and logical operations** in the DFG into groups so that the operations in the **same group can be executed concurrently**, while taking into consideration possible trade-offs between the total execution cost and hardware cost.
- A group of **concurrent computations** to be **executed simultaneously** is referred to as a control step.
- The total number of **control steps** needed to execute all operations in the DFG, the minimum number of functional units of each type to be used in the design and the **lifetimes of the variables** generated during the computation of operations are determined in the scheduling step.

Selection or Allocation

- Selection is the process of **choosing resources** from the **library**, which involves tradeoffs according to different features like **delay**, **area**, **power** and **leakage**.
- **Resource allocation** is the process of determining the number of functional units of each type for performing operations, memory units (registers) for storing data values and interconnects for data transportation.
- Often, the selection and allocation processes are a single task.
- Allocation is further divided into sub-tasks, such as
 - Functional unit allocation,
 - Memory unit allocation and
 - Interconnect allocation.
- Resource **allocation** and **binding** may share resources so that the same hardware can be used to execute different operations or so that the same register can be used to store more than one variable.

Binding or Assignment

- **Binding** or assignment is the process of assigning variables to memory units and data transfers to interconnections.
- Binding is further divided into several sub-tasks, such as
 - Functional unit binding,
 - Memory unit binding and
 - Interconnect binding.
- Functional unit binding involves the mapping of operations in the behavioral description into a set of selected functional units.
- Memory unit binding maps data carriers (constants, variables, arrays) in the behavioral description onto storage elements (read-only memories, registers, memory units) in the data path.
- The interconnect binding task maps every data transfer in the behavior onto a set of interconnection units for data routing.

Output Generation

- In the output generation phase, design output is generated.
- The output should be in a form such that logic-level synthesis tools can optimize the combinational logic and layout synthesis tools can design the chip geometry.
- The generated output is generally in a low-level HDL, such as structural VHDL.
- A demonstrative **example**: A small synthesis example is presented to demonstrate the various phases of behavioral synthesis in detail.
- Suppose that we want to synthesize hardware to perform the operation

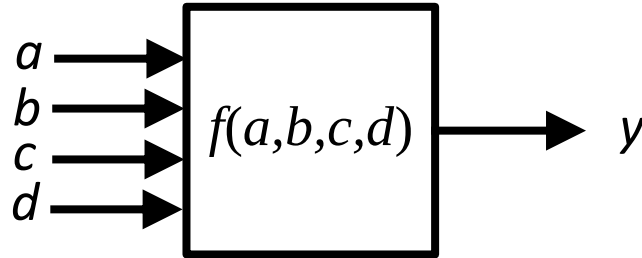
$$y = (a + b) \times (c - d)$$

- The steps can be seen in the self-explanatory in the next few slides.

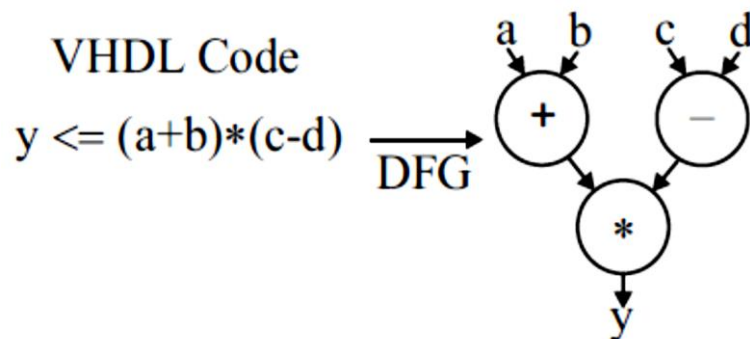
Example:

- Let have a look at a small design example to demonstrate the various phases of design steps in detail.
- Suppose that we want to implement a hardware to perform the operation

$$y = (a + b) \times (c - d)$$

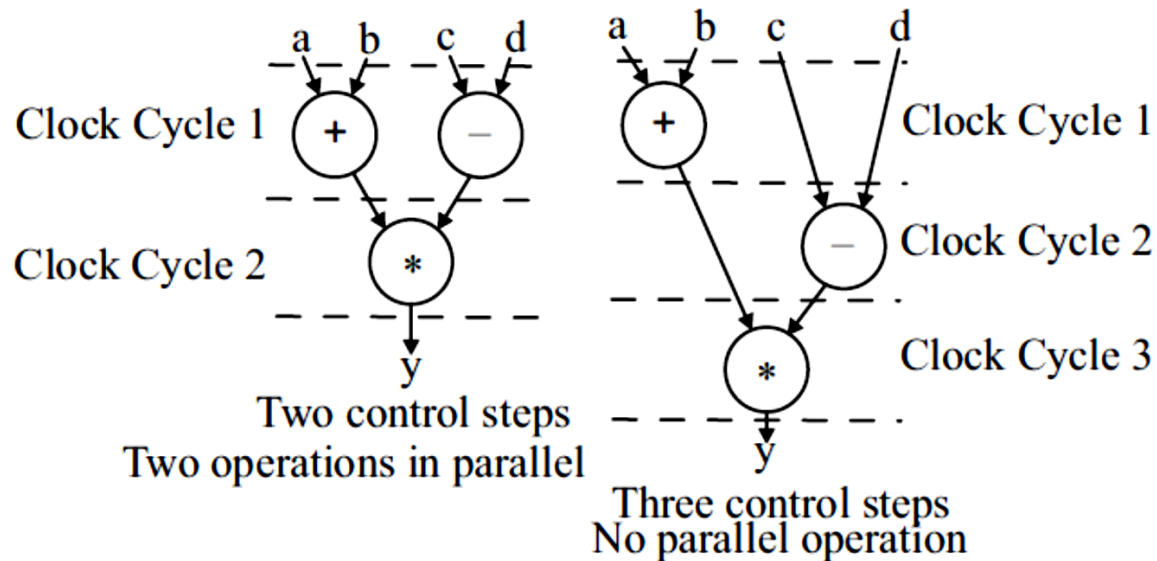


Example:



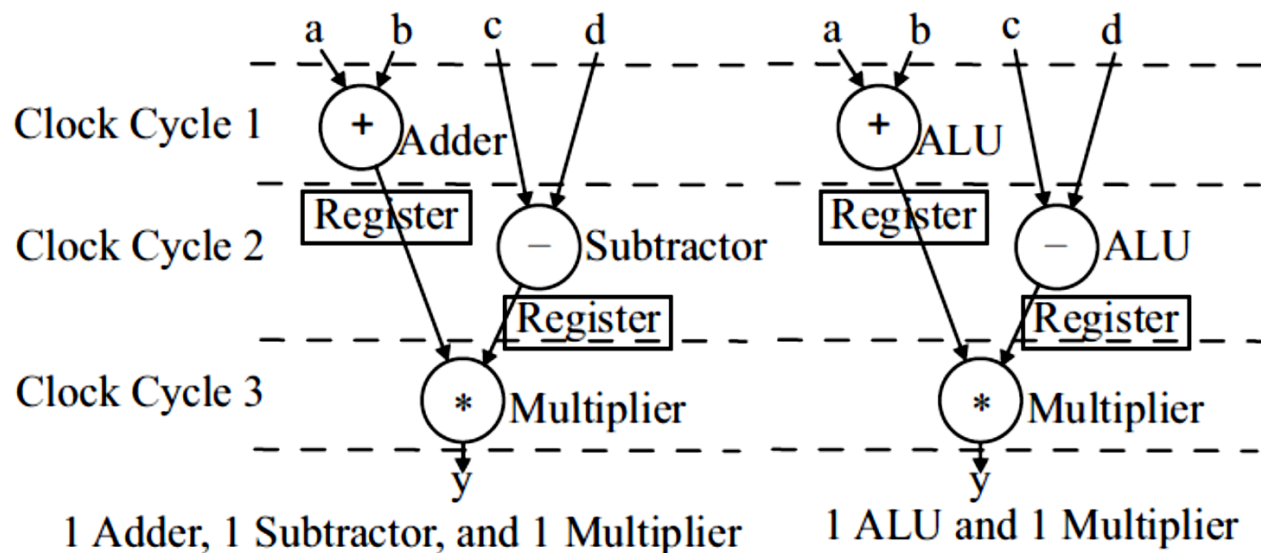
(a) Step 1: Compilation and transformation

Example:



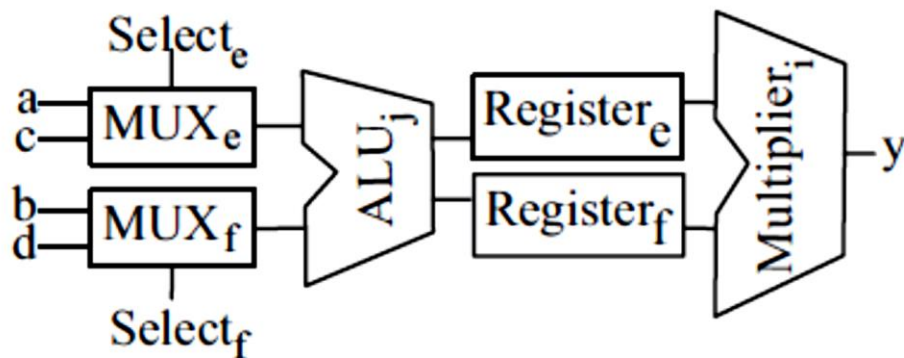
(b) Step 2: Scheduling (time or resource constraints)

Example:



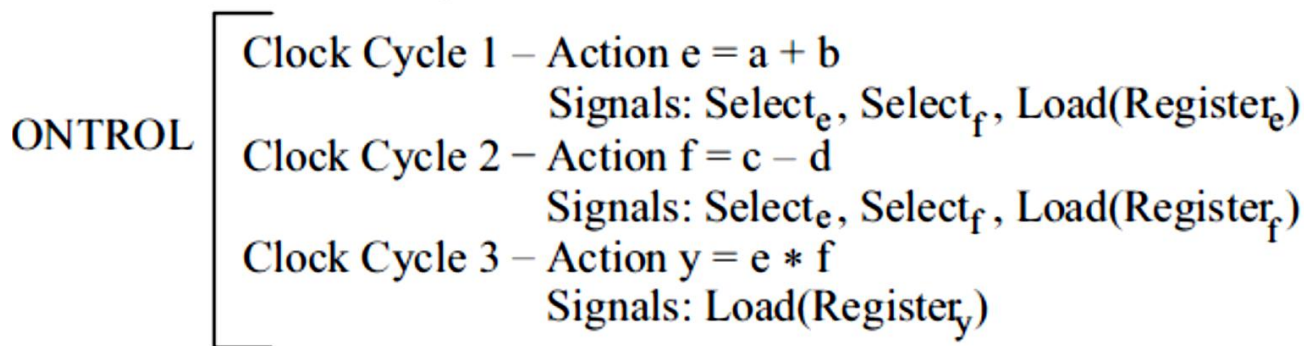
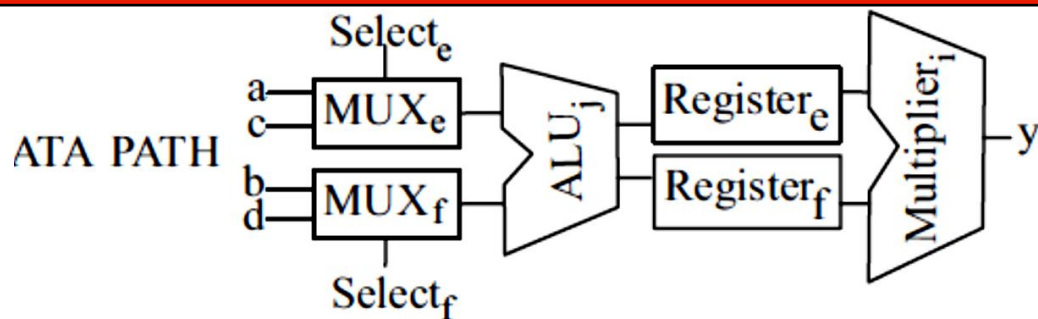
(c) Step 3: Allocation (fixed amount and types of resources)

Example:



(b) Step 5: Connection allocation (communication between resources: bus, buffer, or MUX)

Example:



(c) Step 6: Architecture generation (data path and control)

Behavioral HDL to CDFG Translation or Compilation

- The translation or compilation phase of high-level synthesis is the process by which a behavioral specification is translated to an **intermediate representation**.
- This is the first step of high-level synthesis; however, this step often does not attract much attention because tools perform the task without the intervention of the user, but it is crucial and results in an intermediate representation that is **essential for subsequent synthesis**.
- There are many such intermediate representations including **data flow graph (DFG)**, **control flow graph (CFG)** and **control data flow graph (CDFG)**. Each of these has its own advantages.
- For **example**, although the DFG is simple, the CDFG provides better options for **design space exploration**. The need for a CDFG and a DFG in high-level synthesis and **hardware/software co-synthesis** has been emphasized.
- A representation called sequencing DFG, which is a **directed acyclic graph**, is a simpler form of CDFG. For circuits with a balanced mix of data and control flow, a **hierarchical conditional dependency graph (HCDG)** is often used.

Behavioral HDL to CDFG Translation or Compilation

- An example of a CDFG is presented in the **example** as outlined in the next slide.
- The formal definitions of DFG and CFG are as follows:

A **DFG** is a **directed graph** $G = (V, E)$, where $V = v_1, v_2, \dots, v_n$ is a finite set whose elements are “**nodes**” and $E = V \times V$ is an asymmetric “**dataflow relation**”, whose elements are called “**data edges**”.

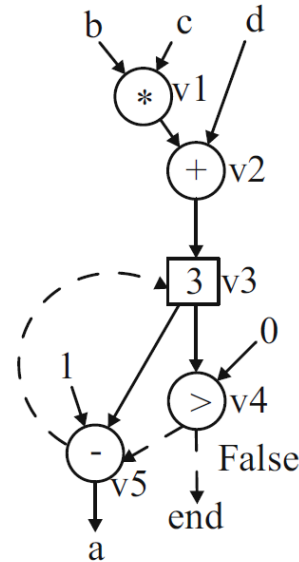
A **CFG** is a **directed graph** $G = (V, E)$, where $V = v_1, v_2, \dots, v_n$ is a finite set whose elements are “**nodes**” and $E = V \times V$ is a “**control flow relation**” whose elements are directed “**sequence edges**.”

Behavioral HDL to CDFG Translation or Compilation

→ Control and DFG example

```
a := b * c + d;  
while (a > 0) do  
    a := a - 1;  
end while;
```

VHDL Description



CDFG Representation

Operational node: v1, v2, v5,
Storage node: v3
Control node: v4

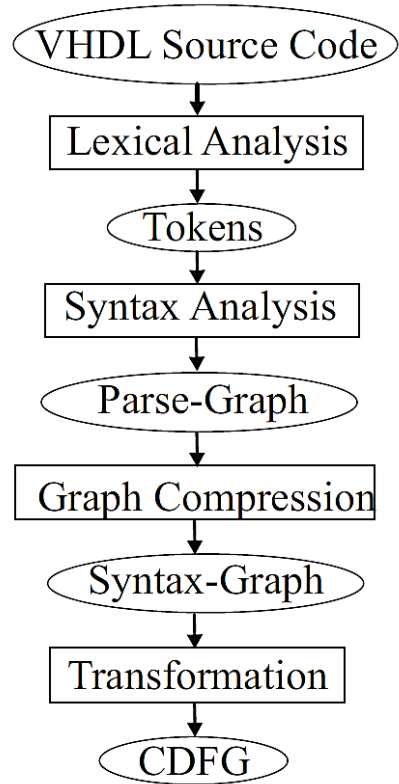


Behavioral HDL to CDFG Translation or Compilation

- The steps for converting the VHDL description to a CDFG are presented in next slide.
- The steps are similar to the steps of a **high-language compiler** used in standard computers.
- The source program is translated into a **stream of tokens** in the **lexical analysis phase**.
- Each token is a sequence of characters with collective meaning, such as an **identifier**, a **keyword**, an **operator** or a **punctuation** character.
- The **syntax analysis phase** verifies the syntax by imposing a **hierarchical structure** on the stream of tokens.
- Compiler parses tokens and generates a parse tree using explicit codes. The **parse tree** is then compressed to obtain a **syntax tree** in which the operators appear as the interior nodes and the operands of an operator are the children of the node for that operator.
- Finally, the syntax tree is transformed to obtain the overall CDFG and DFG representation of the input VHDL.

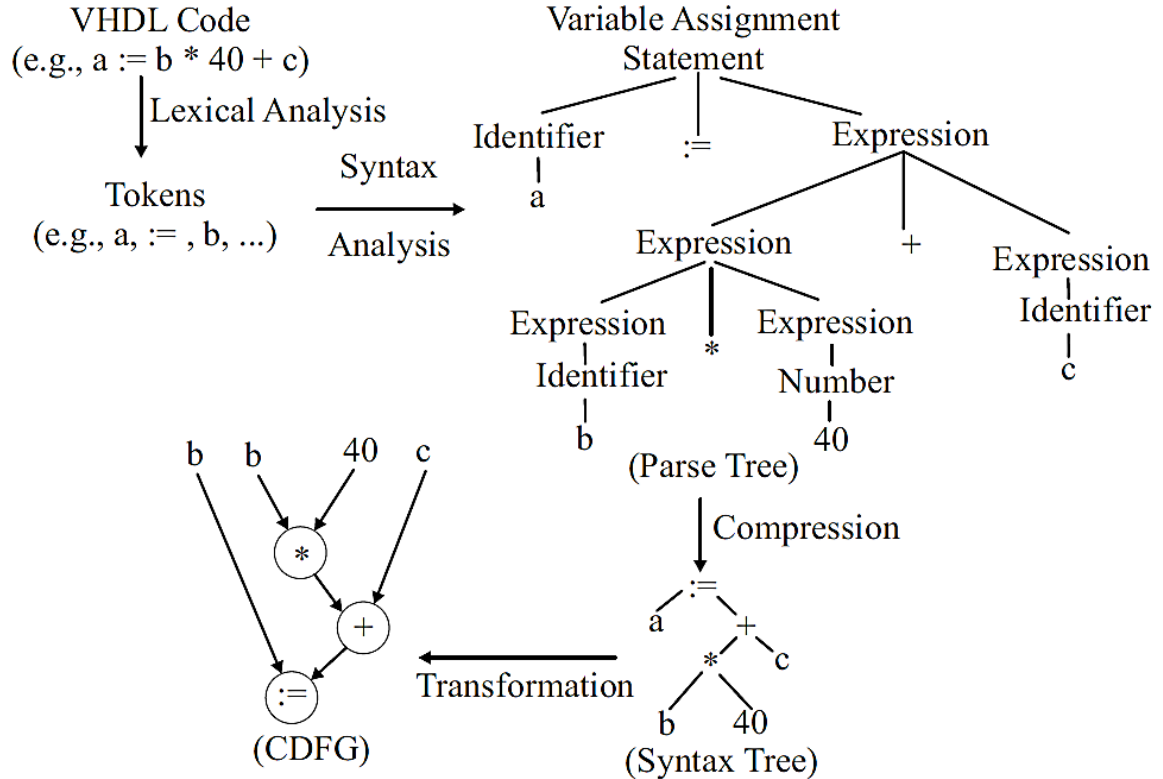
Behavioral HDL to CDFG Translation or Compilation

→ Steps for converting VHDL to CDFG



Behavioral HDL to CDFG Translation or Compilation

- The example represented in the figure demonstrates the VHDL to CDFG transformation. It may be noted that translators and compilers only support a subset and not all features of VHDL.
- VHDL to CDFG conversion example



Scheduling Algorithms

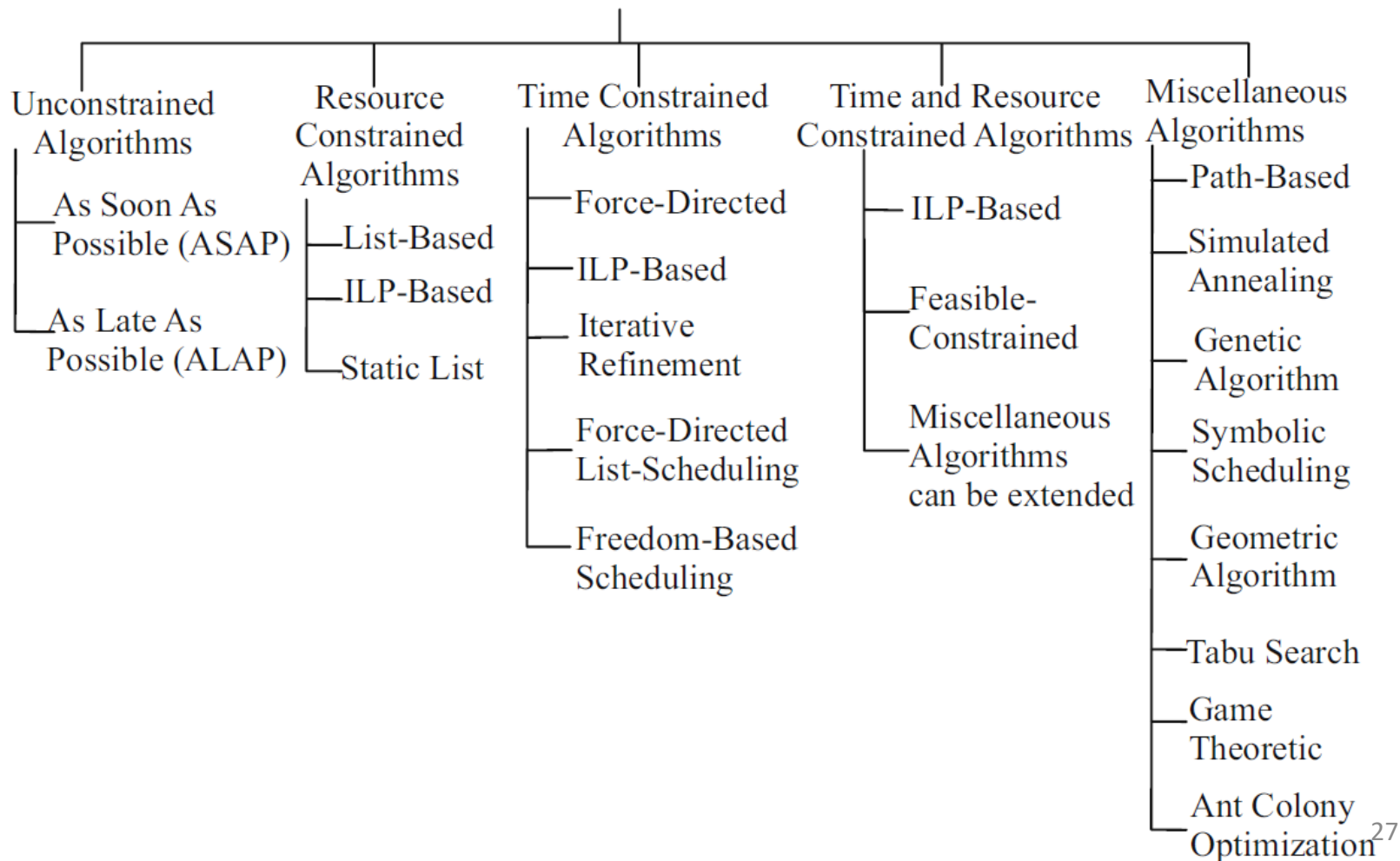
- Scheduling algorithms are the unseen maestros, conducting the intricate symphony of operations that bring digital circuits to life with precision and harmony.



Scheduling Algorithms

- The **scheduling** problem is a **non-deterministic polynomial (NP)** problem.
- **Behavioral scheduling algorithms** may be of various types based on the constraints and optimization schemes, as shown in next slide.
- Various scheduling algorithms are described in.
- The commonly used scheduling techniques are integer linear programming, **as soon as possible (ASAP)**, **as late as possible (ALAP)**, **list-based** scheduling, **force-directed** scheduling and **freedom-based** scheduling.
- The algorithms use power, leakage, area, etc. as **objective functions** and time, resources and area as **constraints**.
- In this section, selected algorithms are discussed generically.

Scheduling Algorithms



ASAP and ALAP Scheduling and Mobility

- The **ASAP scheduling** process arranges the operations topologically according to their data or control flow.
- ASAP scheduling places the operations in the sorted order by stamping them in the **earliest possible control step**.
- The ALAP scheduling process is similar to ASAP; however, it is in the reverse order as that of ASAP.
- The ALAP scheduling places the operations in the **latest possible control step**.
- These two algorithms are simple yet needed in most of the advanced scheduling algorithms.

ASAP and ALAP Scheduling and Mobility

- The ASAP algorithm is **unconstrained**, i.e., it considers neither resource nor time constraints for scheduling.
- The ALAP scheduling considers the number of steps resulting from the ASAP schedule as a **latency constraint**.
- The ASAP and ALAP **time stamps** provide lower and upper bounds on possible control steps for scheduling an operation.
- The difference between the ALAP and ASAP time stamp is called the **mobility of an operation**.
- The time complexity of these algorithms is $O(|V| + |E|)$.
- Algorithms 1 and 2, respectively, present the two algorithms.
- Figs. 12 and 13 demonstrate the algorithms with the help of the **hardware abstraction layer (HAL)** differential equation solver high-level synthesis benchmark.

$$\frac{d^2y}{dx^2} + 3 \frac{dy}{dx} x + 3y = 0$$

```

while (x < a) do
  x1 := x + dx;
  u1 := u - ( 3 * x * u * dx) - (3 * y * dx);
  y1 := y + ( u * dx);
  c = x < a;
  x := x1;
  u := u1;
  y := y1;
end while

```

Example: HAL solver

A numerical solution for a differential equation

```

x = x[n];      x1 = x[n+1];
y = y[n];      y1 = y[n+1];
u = u[n];      u1 = u[n+1];

```

$$u = \frac{dy}{dx} \approx \frac{y[n+1]-y[n]}{dx} \Rightarrow$$

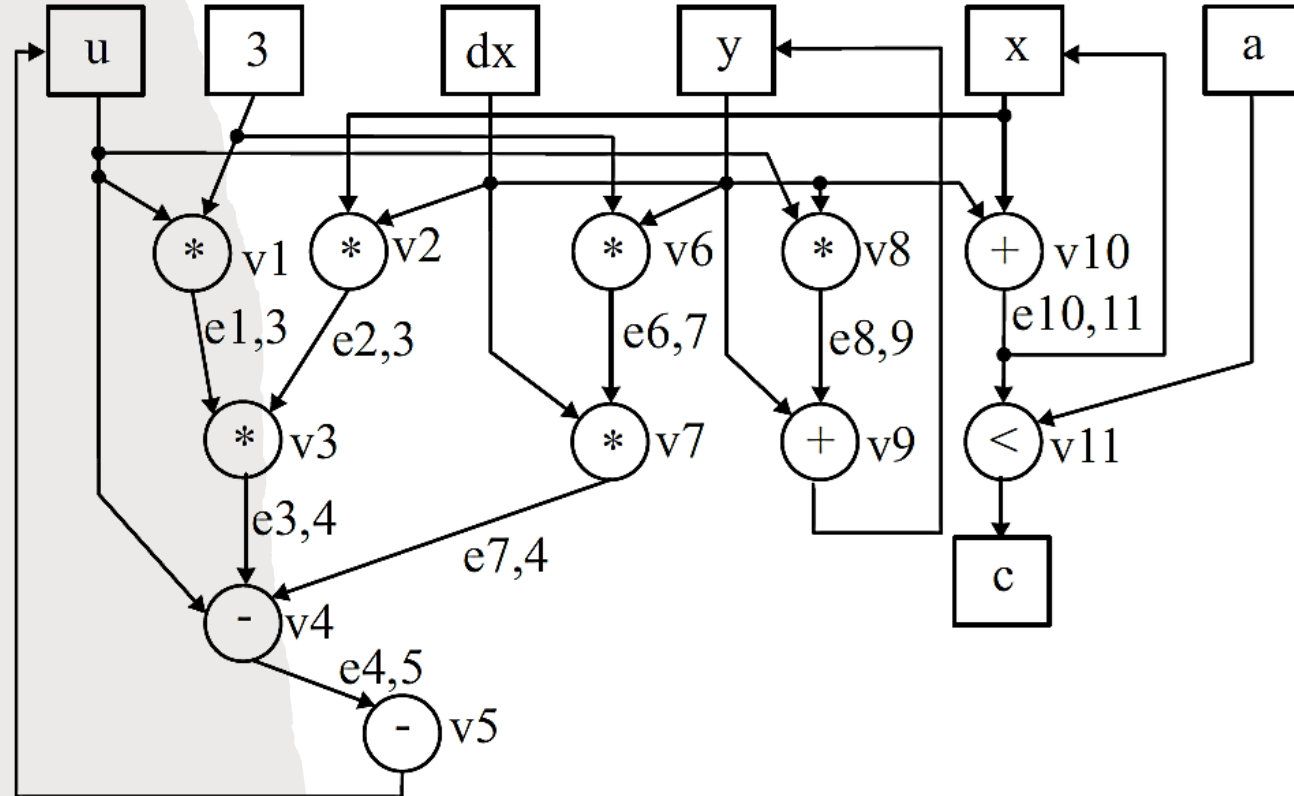
$$y[n+1] = y[n] + u \, dx;$$

$$\frac{d^2y}{dx^2} = \frac{du}{dx} = \frac{u[n+1]-u[n]}{dx};$$

y is calculated for $0 \leq x \leq a$

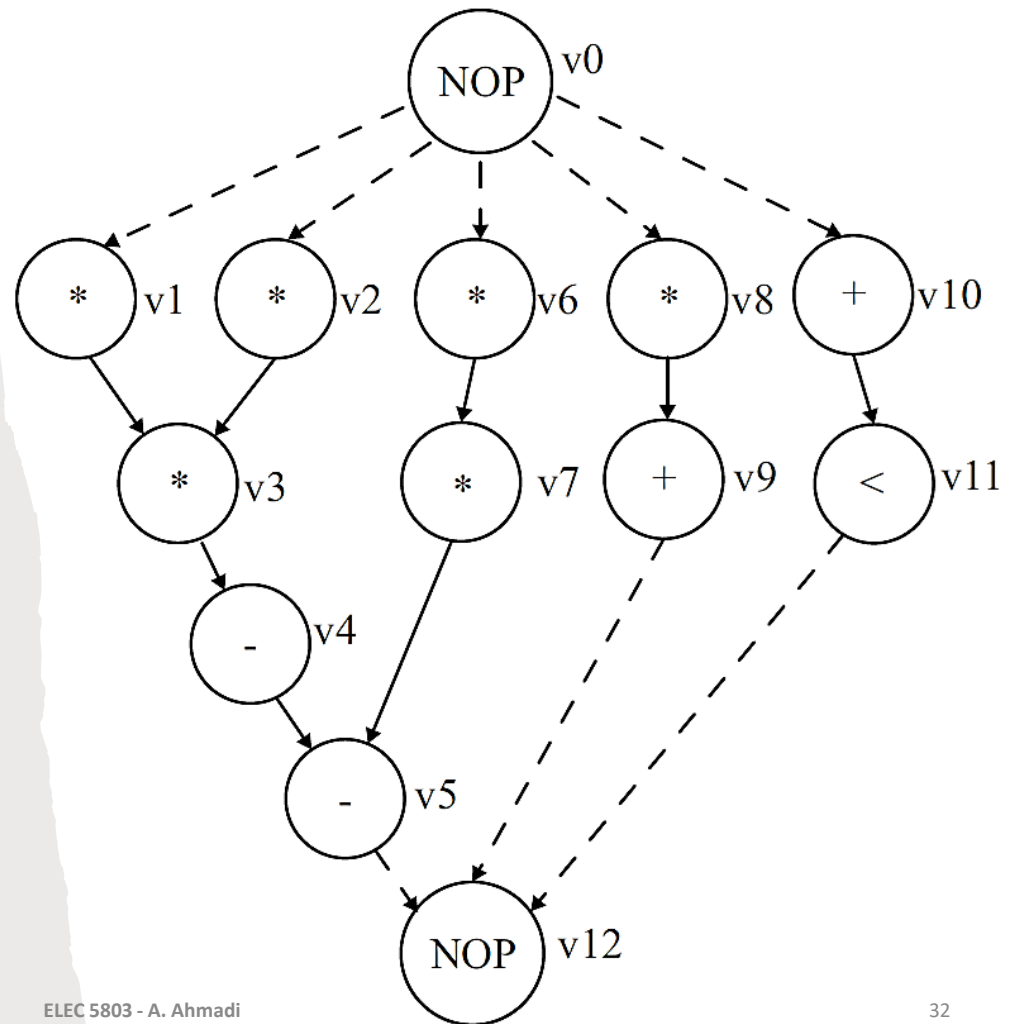
Example: HAL solver

HAL DFG
showing edges
and variables



Example: HAL solver

HAL sequencing DFG



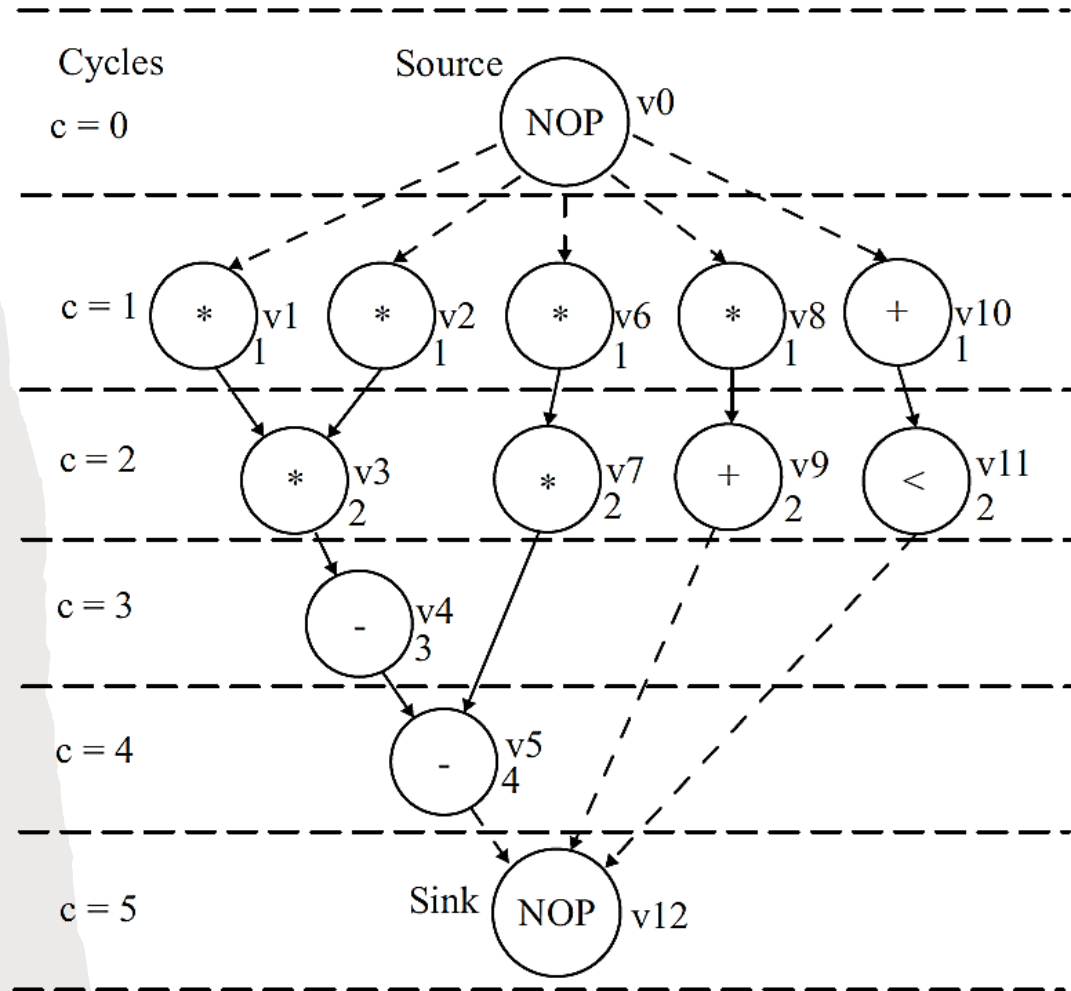
Algorithm 1 As Soon As Possible (ASAP) Scheduling

- 1: For a sequencing DFG $G(V, E)$ time stamp source vertex v_0 with $c_0 = 0$.
 - 2: **while** (Sink vertex v_N is not considered for time stamping) **do**
 - 3: Select a vertex v_i whose all predecessors are scheduled.
 - 4: Schedule v_i by setting its time stamp as c_i maximum start time of all its predecessors plus its delay.
 - 5: **end while**
 - 6: Return schedule of the DFG $\{C_S = c_0, c_1, \dots, c_N\}$.
-

Example: HAL solver

Figure 13:

a. ASAP schedule



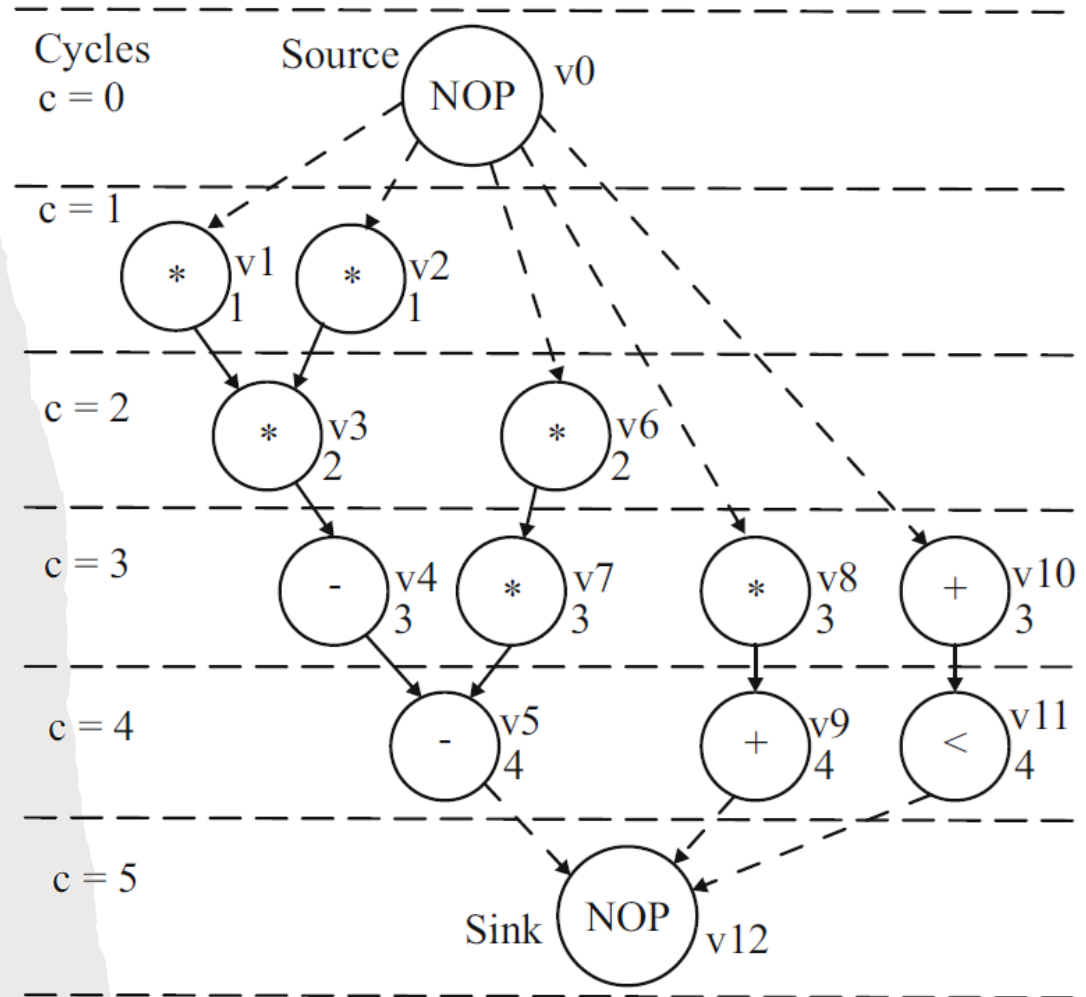
Algorithm 2 As Late As Possible (ALAP) Scheduling

- 1: For a sequencing DFG $G(V, E)$ time stamp sink vertex v_N with $c_N = \text{ASAP Latency Bound} + 1$.
 - 2: **while** (Source vertex v_0 is not considered for time stamping) **do**
 - 3: Select a vertex v_i whose all successors are scheduled.
 - 4: Schedule v_i by setting its time stamp as c_i minimum start time of all its successor minus its delay.
 - 5: **end while**
 - 6: Return schedule of the DFG $\{C_L = c_0, c_1, \dots, c_N\}$.
-

Example: HAL solver

Figure 13:

b. ALAP schedule



Example: HAL solver

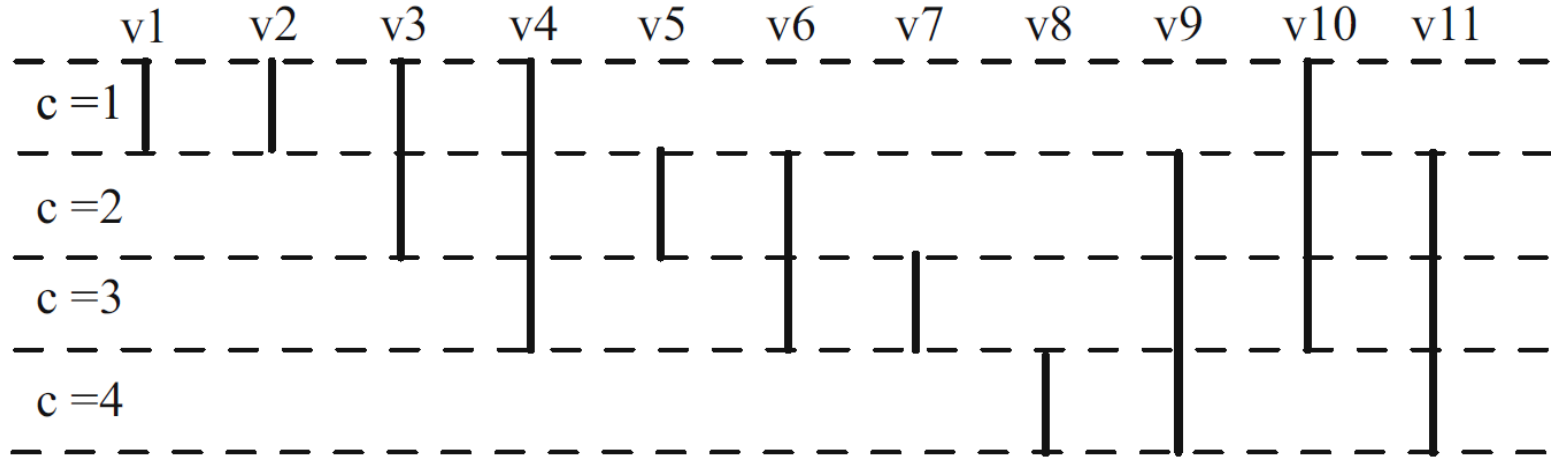


Figure 13.c. Mobility of each vertex

Integer Linear Programming (ILP) Scheduling

- **Integer Linear Programming (ILP)** is a mathematical optimization strategy where a linear objective function is maximized or minimized subject to a set of linear equations and inequalities, with the additional constraint that some or all of the variables are restricted to integer values.
- In the realm of scheduling for behavioral synthesis, ILP is utilized to determine the optimal sequence of operations to achieve goals such as minimal circuit area, power, or delay, while respecting the specific resource constraints and dependencies inherent in an integrated circuit design.
- ILP-based algorithms are the exact solutions of the scheduling problem.
- The ILP-based scheduling algorithm is presented in Algorithm 3.
- The algorithm uses the ASAP and ALAP schedules to impose bounds on the ILP solution space to reduce the search space.
- ILP formulations can be modeled using the AMPL language and solved using various solvers such as LP-Solve.

Algorithm 3 Resource-Constrained ILP-Based Scheduling

- 1: Find ASAP and ALAP schedule of the sequencing DFG.
 - 2: Determine the mobility graph of each vertex.
 - 3: Construct the ILP formulations for the DFG.
 - 4: Solve the ILP formulations.
 - 5: Find the scheduled DFG.
 - 6: Return schedule of the DFG $\{C_{ILP} = c_0, c_1, \dots, c_N\}$.
-

Integer Linear Programming (ILP) Scheduling

- ILP minimizes **cost functions** in the form of **power, area, delay** under resource, time or power constraints.
- The formulation needs the help of a **binary decision** variable that is either 0 or 1.
- The following notations will be used to formulate the ILP:
 1. N_v is the total number of vertex operations in the sequencing DFG, excluding the source and sink nodes (NO-OPs).
 2. v_i is any vertex in the DFG performing certain operations and $1 \leq i \leq N_v$.
 3. FU_k is the functional unit of type k .
 4. M_k is the maximum number of functional units of type FU_k .
 5. $Cost_k$ is cost of functional unit of type FU_k may be power, area or delay.
 6. $C_S[i]$ is the ASAP time stamp for the operation v_i .
 7. $C_L[i]$ is the ALAP time stamp for the operation v_i .
 8. $x_{i,c}$ is the binary decision variable which is 1 if vertex v_i starts in control step c , else 0.

Integer Linear Programming (ILP) Scheduling

1. **Objective function:** The objective is to minimize the total cost corresponding to the DFG over all control steps simultaneously. Using decision variables, one can rewrite the objective function as follows:

$$\text{Minimize: } \sum_c \text{Cost}_{k,v} \times M_k$$

2. **Uniqueness constraints:** These constraints ensure that each operation v_i is scheduled to one unique control step within the mobility range $(C_S[i], C_L[i])$. They are represented as $\forall i, 1 \leq i \leq Nv$,

$$\sum_c x_{i,c} = 1$$

3. **Precedence constraints:** These constraints ascertain that for a vertex v_i , all its predecessors are scheduled at an earlier control step and that its successors are scheduled at a later control step. These are modeled as $\forall i, j, v_i \in \text{Pred}_{oj}$,

$$\sum_{d=C_S[i]}^{C_L[i]} d \times x_{i,d} - \sum_{e=C_S[j]}^{C_L[j]} e \times x_{j,e} \leq -1$$

Integer Linear Programming (ILP) Scheduling

2. Resource constraints: These constraints establish that no control step contains more than M_k operations of type k . These can be enforced as $\forall c, 1 \leq c \leq N_c$,

$$\sum_{i \in F_k} x_{i,c} \leq M_k ,$$

where N_c is the number of clock cycles.

Integer Linear Programming (ILP) Scheduling

- As an **example**, let us assume **four types of resources**: multiplier, adder, subtractor and comparator.
- Let us also assume that the **cost** of a multiplier is 2 units and that the cost of the other units is 1 unit.
- Let us assume that **two** instances of a **multiplier** and **one** instance of the **other resources** are available.
- Let us consider the ASAP schedule, ALAP schedule and the mobility graph shown in Fig. 13(a), (b), (c), respectively, for the HAL DFG.

Integer Linear Programming (ILP) Scheduling

→ The following solution is obtained:

$$x_{1,1} = x_{2,1} = x_{3,2} = x_{4,3} = x_{5,2} = x_{6,3} = x_{7,3} = x_{8,4} = x_{9,4} = x_{10,2} = x_{11,4} = 1$$

→ and the rest $x_{i,j} = 0$.

→ The corresponding scheduled DFG is presented in Figure 14.

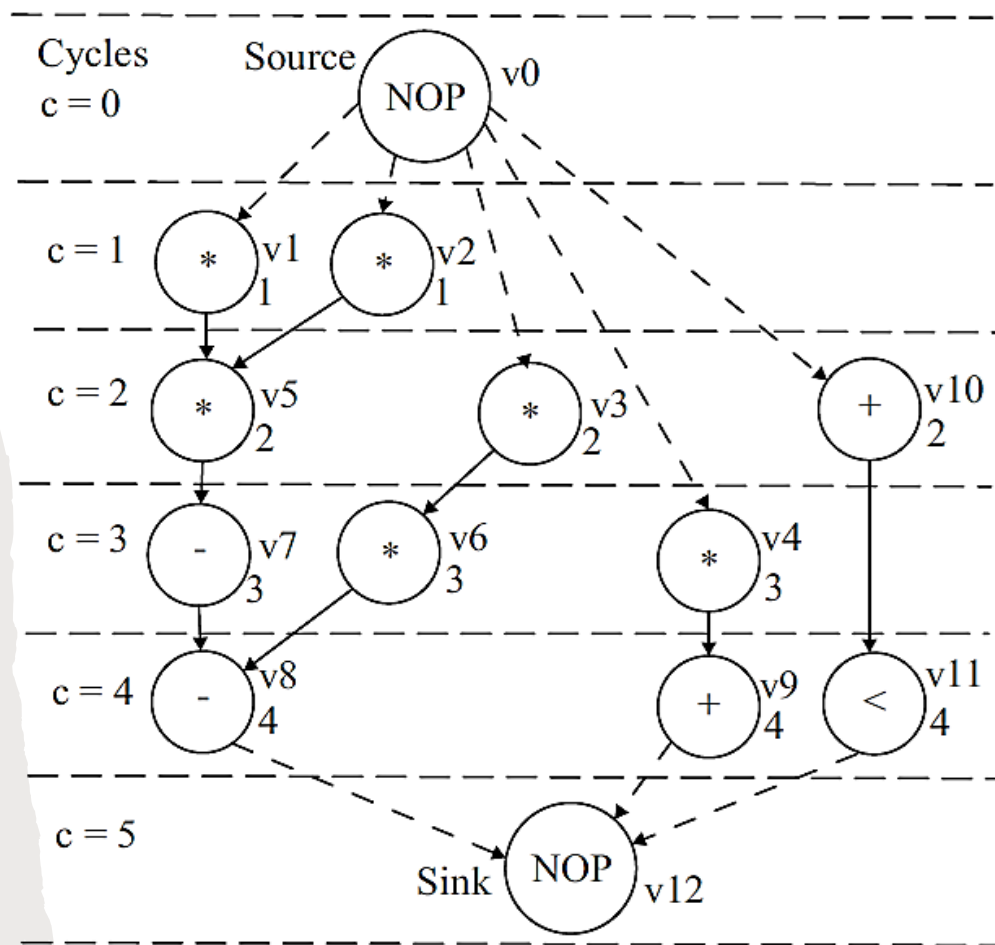
→ The ILP-based algorithm provides an exact solution, but it is slow and has an exponential worst-time time complexity.

→ It is difficult to use it for large and practical circuits as the formulation grows exponentially with the number of vertices.

→ When the number of variables in an ILP formulation reaches around 200, a result is not obtained even after several days of computation.

Integer Linear Programming (ILP) Scheduling

Schedule resulting using the ILP-based
algorithm



List-Based Scheduling (LBS)

- **List-Based Scheduling (LBS)** algorithm is primarily **resource-constrained** but can be modified to accommodate **time constraints**.
- The problem of minimizing latency under resource constraints or minimizing resources under latency constraints is intractable.
- **List-based scheduling** is essentially a **heuristic approach** to solve the scheduling problem.
- A **heuristic approach** refers to a problem-solving or decision-making strategy that **sacrifices optimality for efficiency** by leveraging practical rules of thumb, intuition, or simplifying assumptions.
- Heuristics guide the search for solutions or decisions that are "**good enough**" given limited time, resources, or computational capacity, especially in complex or time-sensitive scenarios.

List-Based Scheduling (LBS)

- The **list-based algorithm** takes a **sequencing DFG** and **resource constraints** as inputs and generates a **scheduled sequencing DFG** as output.
- In list scheduling, the operations available for scheduling are kept in a list for each control step.
- This list is ordered by some **priority function**: mobility of the vertex or the length of path from the operation to the sink while ranking the vertices in decreasing order.
- An operation on the list is scheduled one by one if the resource needed by the operation is free; otherwise, it is deferred to the next clock cycle.
- Algorithm 4 shows the pseudocode of resource-constrained list-based scheduling that minimizes latency.
- Figure 15 shows the scheduled DFG result for a resource constraint of three multipliers and one ALU.

List-Based Scheduling (LBS)

Algorithm 4 Resource-Constrained List-Based Scheduling to Minimize Latency

```
1: Initial clock cycle  $c = 1$ .
2: while (For a sequencing DFG  $G(V, E)$  sink vertex  $v_n$  is not considered for time stamping) do
3:   for all (Resource of type  $k$ ) do
4:     Determine list of vertices of type  $k$  whose predecessors are already scheduled to
       finish execution in  $c$ ,  $L_{F_c, k}$ .
5:     Determine list of vertices of type  $k$  that started at an earlier cycle but whose exe-
       cution is not finished in  $c$ ,  $L_{U_c, k}$ .
6:     Select the vertices from list  $L_{F_c, k}$  such that the resource needed by these vertices
       and  $L_{U_c, k}$  do not violate resource constraints.
7:     Time stamp the above selected vertices at clock cycle  $c$ , i.e.,  $c_i = c$ .
8:   end for
9:   Increment the clock cycle  $c$ .
10: end while
11: Return schedule of the DFG  $\{C_{LBS} = c_0, c_1, \dots, c_N\}$ .
```

List-Based Scheduling (LBS)

→ The **time complexity** of list-based algorithm algorithm is

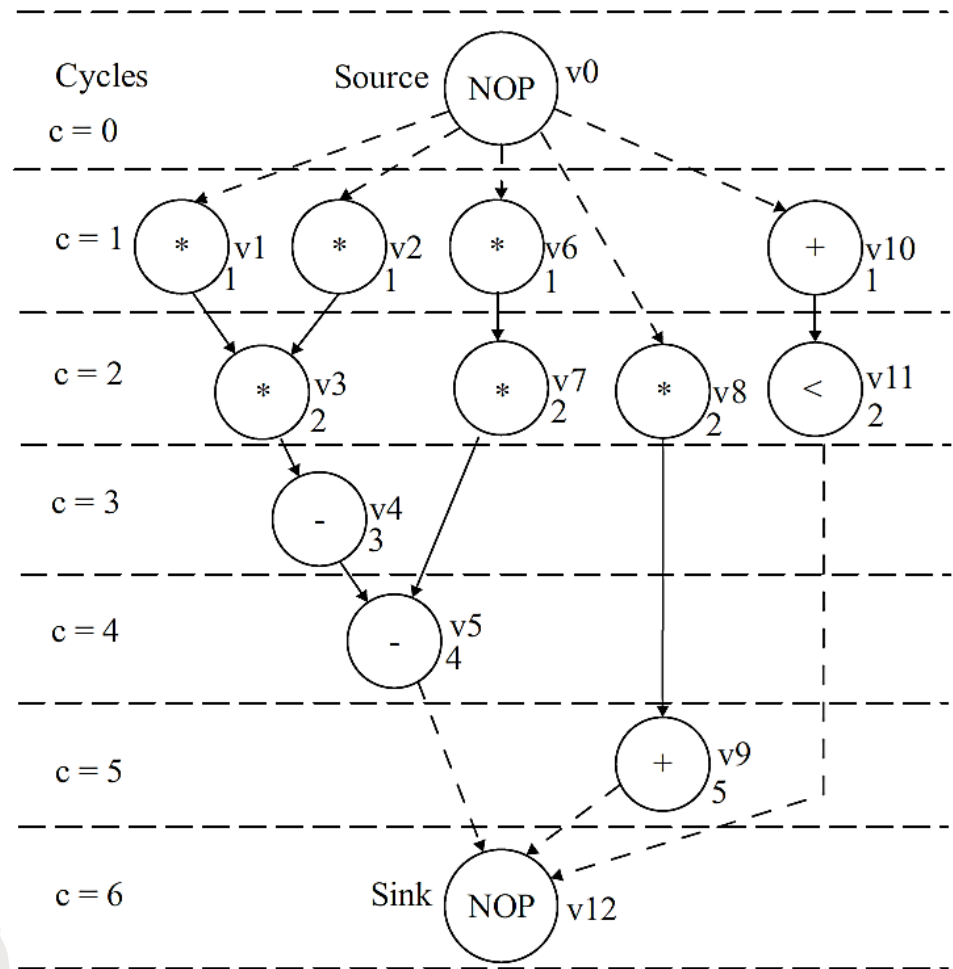
$$O(|E| + |V| \log(|V|) + |V|T_0),$$

where $O(|V|T_0)$ is the worst-case time for creating the list and selecting its first element.

- This algorithm has better time complexity compared to force-directed scheduling and guarantees results.
- The time **complexity of force-directed** scheduling is $O(|V|^3)C_{max}$, where C_{max} is the time constraint expressed in terms of the number of clock cycles.
- Thus, **list scheduling is a better choice for large and practical designs**.

List-Based Scheduling (LBS)

Schedule resulting by using a list-based scheduling algorithm assuming unit delay for each operation with a resource constraint of three multipliers and one ALU



Force-Directed Scheduling (FDS)

- Force-directed scheduling is a heuristic algorithm that can consider both resource and time constraints.
- The basic idea of this algorithm is to balance the concurrency of operations without increasing the total execution time to maximize the utilization of resources such that the number of required resources is minimal.
- The algorithm is iterative; that is, one operation is time-stamped in each iteration, and selection of the time stamp is made to achieve balance of distribution of operations.
- The term “force” has originated from a mechanical analogy; i.e., force is the product of a spring constant and displacement. This is motivated by Hooke’s law,
$$\text{Force} = \text{Spring constant} \times \text{Displacement.}$$
- The algorithm is presented with the help of pseudocode in Algorithm 5.

Algorithm 5 Force-Directed Scheduling (FDS)

- 1: **while** (All vertices v_i in V are not considered for time stamping) **do**
 - 2: Find ASAP schedule C_S .
 - 3: Find ALAP schedule C_L .
 - 4: Determine mobility of each operation from C_S and C_L .
 - 5: Update distribution graphs (DG) for each resource type.
 - 6: Calculate self-forces for every feasible clock cycle.
 - 7: Add predecessor and successor forces to self-forces to calculate total force.
 - 8: Schedule the operation with lowest total force by setting the time stamp c_i as the selected clock cycle c and adjust the mobility.
 - 9: **end while**
 - 10: Return schedule of the DFG $\{C_{FDS} = c_0, c_1, \dots, c_N\}$.
-

Force-Directed Scheduling (FDS)

- The algorithm starts with the computation of the **mobility** of each operation based on its **ASAP** and **ALAP** time stamps.
- An operation is **time-stamped** for a specific clock cycle for which the **total force is minimum**.
- The total force is the sum of two forces, the **self-force** and **mutual force**, due to predecessors and successors.
- For vertex v_i to be considered for control step or clock cycle c has the total force:

$$Total\ Force\ (i, c) = Self_Force\ (i, c) + Mutual\ Force\ (i, c)$$

- The **self-force** of an operation or vertex is the set of forces the operation is subjected to while tentatively scheduled in a control step in its mobility range.

Force-Directed Scheduling (FDS)

- For a vertex v_i needing a resource type k in clock cycle c , the self-force can be expressed as:

$$Self_Force(i, c) = \sum_k Probability(k, c) - \frac{1}{m_i + 1} \sum_{c^*=C_S[i]}^{C_L[i]} Probability(k, C^*).$$

- Here $Probability(k, c)$ is the sum of probabilities of the operations that can be implemented by a specific resource of type k in the type distribution.
- **Type distribution** is indicative of concurrency of similar types of operations, the plot of which is called **distribution graph** (DG).
- The **mobility of each operation** v_i is

$$m_i = CL[i] - CS[i]$$

- and mobility range is

$$[CS[i], CL[i]].$$

Force-Directed Scheduling (FDS)

- The **mutual force** due to predecessor/successor forces is due to the **operation dependencies**.
- This force arises when a tentative assignment of an operation to a control step or clock cycle restricts the mobility of its predecessors and successors. For a vertex v_i needing a resource of type k in clock cycle c , the mutual force can be expressed as follows:

$$Self_Force(i, c) = \sum_k Probability(k, c) - \frac{1}{m_i + 1} \sum_{c^*=C_S[i]}^{C_L[i]} Probability(k, C^*).$$

- where $[C_S[i], C_L[i]]$ is the initial mobility range and $[C_S^+[i], C_L^+[i]]$ is the restricted mobility range.
- The **time complexity** of force-directed scheduling is $O(|V|)3C_{max}$, where C_{max} is the time constraint expressed in terms of number of clock cycles.
- The cubic time complexity makes it more time consuming for large designs and hence has reduced its role in solving practical designs which are often large. Thus, **list-based scheduling may be a better choice**.

Game Theory Scheduling (GTS)

- **Game theory** also has been explored as a solution approach for behavioral scheduling. Game theory has been used by economists to understand the action of economic agents by analyzing the interaction of decision makers with conflicting objectives. In game theory, a game can be either cooperative or non-cooperative depending on whether an agreement has been made before.
- In the context of scheduling, GTS aims to allocate resources and schedule tasks based on the concept of **strategic interaction** between different entities, such as processes, machines, or agents. Key concepts of this algorithm are:
 1. **Strategic Decision-Making**: GTS encompasses the idea that entities involved in the scheduling process act strategically to optimize their **individual objectives**, while considering the decisions of others and their impact on the **overall efficiency**.
 2. **Resource Allocation as a Game**: GTS treats the scheduling problem as a "**game**" in which entities make decisions to optimize **their own performance** taking into account the choices made by other entities. This can lead to mutually beneficial or **competitive interactions** affecting scheduling outcomes.
 3. **Optimization through Strategic Interaction**: GTS leverages game-theoretic concepts, such as equilibrium, to model and analyze the strategic interactions and decisions made by entities, with the goal of optimizing scheduling performance metrics such as make span, resource utilization, or fairness.

Game Theory Scheduling (GTS)

→ The notable and unique features of game theory that can facilitate associated problems of high-level synthesis are as follows:

1. **Rationality**: Each player always selfishly tries to maximize its gain, which can be suitable to meet conflicting objectives of synthesis.
2. **Coalition**: An important feature that can assist resource allocation during high-level synthesis is coalition formation when a subset of players having the same agenda in terms of strategies can pool their resources.
3. **Competition**: Multiple decision makers control a specified set of variables and seek to optimize their conflicting objectives, a natural fit for behavioral synthesis.
4. **Equilibrium**: When all the objectives of all players are met, a solution is in an equilibrium state, thus ensuring the convergence of solutions.

Game Theory Scheduling (GTS)

- The scheduling problem is modeled as a **first-bid sealed auction** and solved using a game theory approach.
- The auctioning of items by a seller through bidding can be extended to the auctioning of operations in the sequencing unscheduled DFG to the available resources with an operation being bought by only one resource and the sale entity being the cost of execution, with cost being area, power or delay.
- In the presence of multiple operations and resources, an equilibrium point needs to be achieved to converge to a solution.
- An auction consists of a set of available resources M_k (of type k) and a set of interested buyers V (operations or vertices).
- A buyer ($v_i \in V$) may also have some preference for some resources over the others, and this can be specified in terms of a cost function $Cost_k$, in which v_i needs an instance of resource of type k .

Game Theory Scheduling (GTS)

- If the cost of two instances of the same type of resource is needed by an operation in vertex v_i , then obviously the instance is chosen for which the cost is lower.
- In other words, if FU_k^a and FU_k^b are two instances of FU_k and the corresponding costs are $Cost_k^a$ and $Cost_k^b$, respectively, then FU_k^a is chosen if $Cost_k^a < Cost_k^b$; otherwise, FU_k^b is chosen.
- A scheduling algorithm using game theory is presented in Algorithm 6.
- For N_p players, a game with S strategies and a number of clock cycles N_c , the time complexity of the game theoretic-based scheduling algorithm is $O(N_c S N_p^S)$.
- The game theory-based formulations can be solved by the Gambit software.

Algorithm 6 Game Theory Scheduling (GTS) Algorithm

- 1: Initialize control step $c = 0$.
 - 2: Assume the initial scheduled as the ASAP schedule C_S .
 - 3: **while** (Sink vertex v_n is not scheduled) **do**
 - 4: Perform breadth first search (BFS) of current DFG.
 - 5: Create a list of all vertices $List_v$ from current DFG having same time stamp.
 - 6: **for all** (Each set of vertices needing type k resource in cycle c , i.e., $List_v^k$) **do**
 - 7: Calculate cost of the schedule using payoff matrix for $List_v^k$ and M_k at step c .
 - 8: Find Nash equilibrium solution at control step c .
 - 9: Assign vertices in step c that form the Nash equilibrium.
 - 10: $c = c + 1$.
 - 11: **end for**
 - 12: **end while**
 - 13: Return schedule of the DFG $\{C_{GTS} = c_0, c_1, \dots, c_N\}$.
-

Other Algorithms

- Tabu Search Scheduling (TSS) (see **Algorithm 7**)
- Simulated Annealing Scheduling (SAS) (see **Algorithm 8**)
- Genetic Algorithm Scheduling (GAS) (see **Algorithm 9**)
- Ant Colony Scheduling (ACS) (see **Algorithm 10**)
- Automata-Based Symbolic Scheduling¹

1: Haynal, Steven Paul. *Automata-based symbolic scheduling*. University of California, Santa Barbara, 2000.

Algorithm 7 Tabu Search Scheduling (TSS) Algorithm

```
1: Initialize iteration counter Counter = 0.
2: Determine ASAP and ALAP schedules.
3: Generate initial feasible solution  $S_i$ .
4: while (Counter < Max_Counter) do
5:     Counter = Counter + 1.
6:     Generate neighborhood  $S_i^*$ .
7:     if ( $S_i$  is not visited in the previous iterations) then
8:         if ( $S_i^*$  is a better solution than  $S_i$ ) then
9:             return  $S_i \leftarrow S_i^*$ .
10:        else
11:            return Discard the solution  $S_i^*$ .
12:        end if
13:    end if
14: end while
15: Return schedule of the DFG  $\{C_{TSS} = c_0, c_1, \dots, c_N\}$ .
```

Algorithm 8 Simulated Annealing Scheduling (SAS) Algorithm

- 1: Determine ASAP and ALSP schedules.
 - 2: Determine the mobility graph.
 - 3: Initialize iteration counter Counter = 0.
 - 4: Initial feasible solution S as the ASAP schedule.
 - 5: Determine initial $Cost_i$ for the initial solution S_i .
 - 6: Assume initial temperature T as T_i .
 - 7: **while** (Cost is varying) **do**
 - 8: Counter = Maximum number of iterations.
 - 9: **while** (Counter > 0) **do**
 - 10: Generate random transition from S to S^* for a neighborhood solution.
 - 11: Calculate change in cost as: $\Delta_{Cost} = Cost_S - Cost_{S^*}$.
 - 12: **if** ($\Delta_{Cost} < 0$ random(0,1) < $e^{\frac{\Delta_{Cost}}{T}}$) **then**
 - 13: Update the solution with new solution, $S \leftarrow S^*$.
 - 14: **end if**
 - 15: Counter = Counter - 1.
 - 16: **end while**
 - 17: Decrease temperature as: $T = T * Cooling_Rate$.
 - 18: **end while**
 - 19: Return schedule of the DFG $\{C_{SAS} = c_0, c_1, ..., c_N\}$.
-

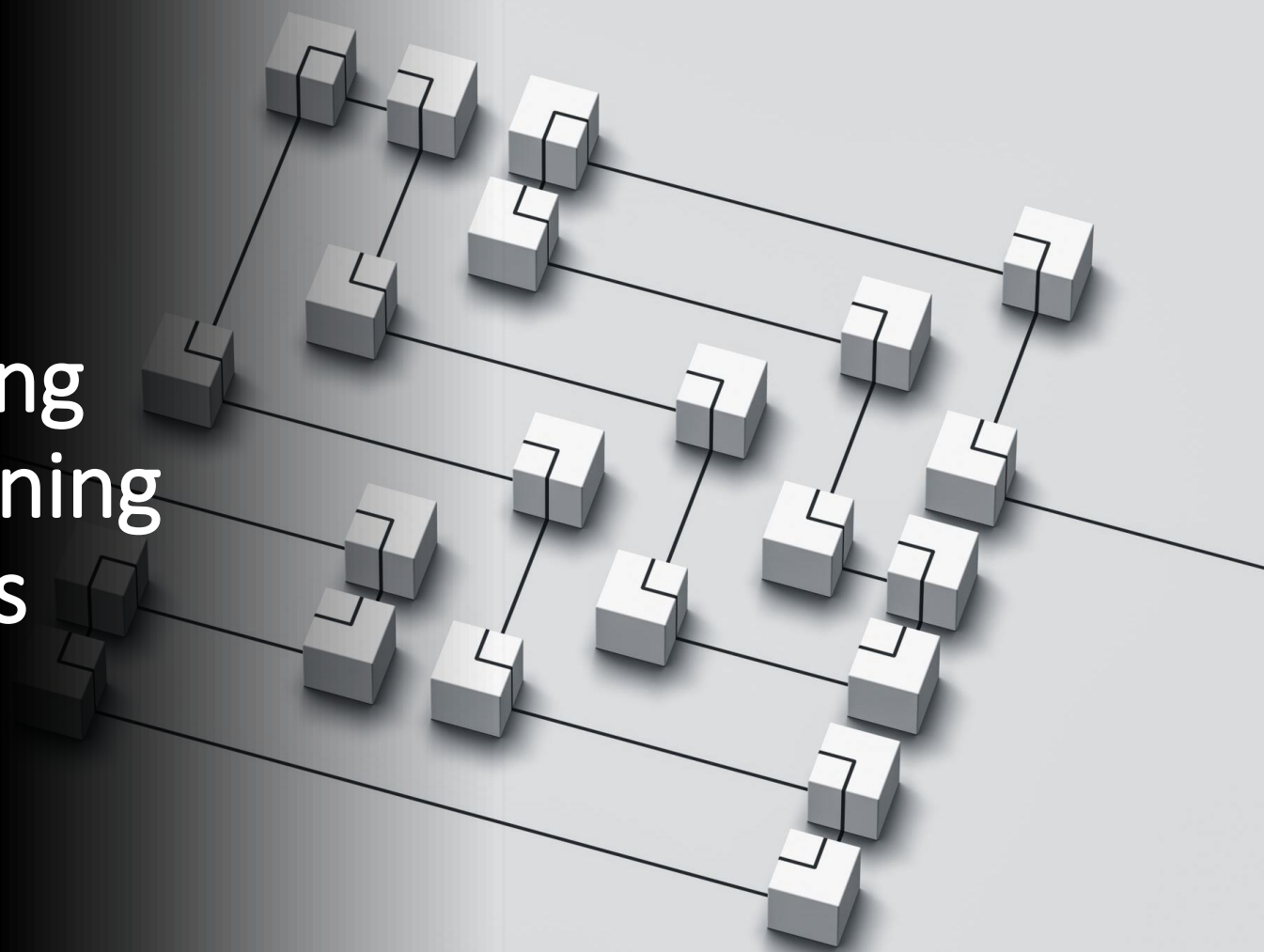
Algorithm 9 Genetic Algorithm Scheduling (GAS) Algorithm

- 1: Initialize iteration counter $i = 0$.
 - 2: Create initial population S_i .
 - 3: **while** (Stopping criterion is not met) **do**
 - 4: $i = i + 1$.
 - 5: **repeat**
 - 6: Select parent's chromosomes from their population based on fitness criteria.
 - 7: Cross over with a high probability: the vertex and resource priority sub-lists for a portion of selected chromosomes, where chromosome is a list containing two sub-lists for vertices and resources in priority order.
 - 8: Mutate with low probability: the vertex and resource priority sub-lists for a portion of selected chromosomes containing vertices and resources sub-lists in priority orders.
 - 9: For all children chromosomes decode them to schedule of vertices using list-based heuristic and evaluate fitness.
 - 10: Create a population S_i by replacing least fit members of population with children chromosomes.
 - 11: **until** (New population size is smaller than the previous population size, i.e., $\text{Size}(S_i) < \text{Size}(S_{i-1})$)
 - 12: **end while**
 - 13: Return best solution S_i as $\{C_{\text{GAS}} = c_0, c_1, \dots, c_N\}$.
-

Algorithm 10 Ant Colony Scheduling (ACS) Algorithm

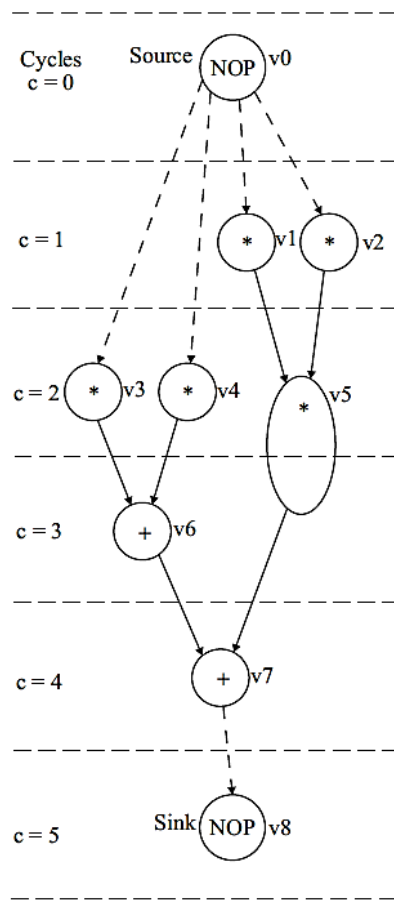
- 1: Initialize parameters such as evaporation ratio, pheromone trails, conditional probability, and evolving bounds.
 - 2: Construct n_{ant} number of ants.
 - 3: Assume an initial solution S .
 - 4: **while** (Stopping criterion is not met) **do**
 - 5: **for** $i = 0$ to n_{ant} **do**
 - 6: i th ant constructs a list of vertices $List_i$ using pheromone trails and mobility.
 - 7: Generate a new schedule S^* using List-based or Force-directed approaches for graph $G(V, E)$, resource constraints, and $List_i$.
 - 8: **if** (S^* is better than S) **then**
 - 9: Update solution with new solution, i.e., $S \leftarrow S^*$.
 - 10: **end if**
 - 11: **end for**
 - 12: Update evolving bounds of the pheromone trails.
 - 13: Update mobility and pheromone trails.
 - 14: **end while**
 - 15: Return best solution S as $\{C_{\text{ACS}} = c_0, c_1, \dots, c_N\}$.
-

Chaining, Multicycling and Pipelining Data Paths

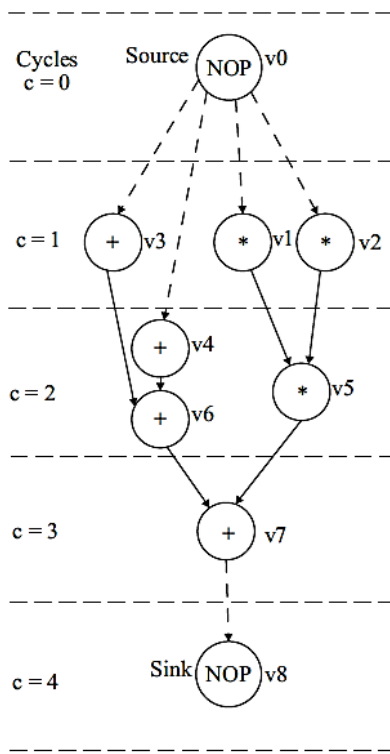


Chaining, Multicycling and Pipelining Data Paths

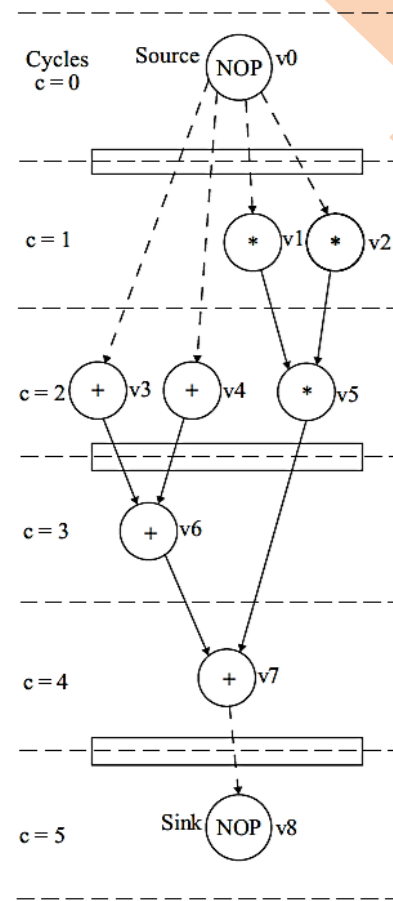
- Because of **data dependencies**, maximum utilization of resources is not possible.
- Data dependencies place **bottlenecks** on concurrent execution of operations even when abundant resources or functional units are available.
- Thus, techniques like **chaining**, **multicycling** or **pipelining** and their combinations are used to improve the optimization of resource utilization. A data path can be designed for **single cycling** and **multicycling** executions.
 - In a **single-cycle** data path operation, each operation is executed in a single cycle with clock width determined by the slowest instruction.
 - In **multicycling** data path, one operation can take more than one clock cycle for its execution. In chaining, more than one operation are executed together in a clock cycle.
 - In a **pipelined** scenario, a data path is segmented into several stages, where each stage is made available for the resources to execute the relevant operations of that stage.
- Figure 16 represents the three different data path scenarios.
- The scheduling and binding problems are more complex for pipelined data paths.
- The above scheduling algorithms can handle these data path specifications.



(a) Multicycling



(b) Chaining



(c) Pipelined